



Projet de calcul parallèle

Simulation d'une épidémie avec OpenMP et Open MPI

Auteurs :

BELKADI Farès
CHENNOUF Ziyad
GAYON Mathis
NADAUD Antonin

Encadrant :

GARCIA Thierry

10 juin 2026

Sommaire

1	Introduction	2
2	Version séquentielle	2
2.1	Représentation des données	2
2.2	Boucle principale	2
2.3	Résultat de référence	3
3	Parallélisation avec OpenMP	3
3.1	Principe	3
3.2	Boucles parallélisées	3
3.3	Compilation et exécution	4
3.4	Mesures obtenues	4
4	Parallélisation avec Open MPI	5
4.1	Mémoire distribuée et initialisation	5
4.2	Découpage de la grille	6
4.3	Lignes fantômes	6
4.4	Échange des frontières	6
4.5	Réduction des résultats	7
4.6	Compilation et exécution	7
5	Analyse comparative	7
5.1	Différences entre OpenMP et MPI	7
5.2	Validation et limites des mesures	8
6	Conclusion	8
7	Références	9

1 Introduction

Ce projet étudie la parallélisation d'une simulation simplifiée de propagation d'une épidémie. La population est représentée par une grille carrée de $N \times N$ cellules. Chaque cellule correspond à une personne dont l'état vaut 0 si elle est saine, 1 si elle est infectée et 2 si elle est guérie.

À chaque itération, une personne saine devient infectée si au moins deux de ses quatre voisins orthogonaux sont infectés. Une personne infectée devient guérie à l'itération suivante, tandis qu'une personne guérie conserve son état. Trois personnes infectées sont placées au centre de la grille au début de la simulation.

L'objectif est d'abord de comprendre les dépendances du programme séquentiel, puis de proposer deux versions parallèles :

- une version OpenMP utilisant plusieurs threads sur une mémoire partagée ;
- une version Open MPI utilisant plusieurs processus et des échanges de messages.

2 Version séquentielle

2.1 Représentation des données

La grille est stockée dans un tableau à une dimension. La macro `IDX(i, j)` transforme les coordonnées (i, j) en un indice linéaire :

```
1 #define IDX(i, j) ((i)*N + (j))
```

Deux tableaux de même taille sont utilisés. Le tableau **A** contient l'état courant de la population et le tableau **B** reçoit l'état de l'itération suivante. Ce double stockage évite qu'une cellule déjà mise à jour modifie le calcul de ses voisines pendant la même itération.

2.2 Boucle principale

La simulation répète le calcul pendant `ITER` itérations. Seules les cellules internes sont mises à jour, car les bords ne possèdent pas quatre voisins.

```
1 for (int it = 0; it < ITER; it++) {
2     for (int i = 1; i < N - 1; i++) {
3         for (int j = 1; j < N - 1; j++) {
4             /* Lecture dans A et ecriture dans B */
5         }
6     }
7 }
```

```

6     }
7
8     int *tmp = A;
9     A = B;
10    B = tmp;
11 }

```

Pour une cellule donnée, le programme lit uniquement $A[\text{IDX}(i, j)]$ et ses quatre voisins. Il écrit le résultat dans une position distincte de B. Les calculs des cellules d'une même itération sont donc indépendants. Cette propriété rend possible la parallélisation de la boucle portant sur les lignes.

2.3 Résultat de référence

Après la dernière itération, le programme compte les personnes saines, infectées et guéries. Ce comptage sert à vérifier les versions parallèles : pour les mêmes valeurs de N et de ITER , elles doivent produire les mêmes compteurs que la version séquentielle.

3 Parallélisation avec OpenMP

3.1 Principe

OpenMP repose sur une mémoire partagée. Tous les threads accèdent donc aux mêmes tableaux A et B. La parallélisation est ajoutée avec des directives `#pragma omp parallel for` devant les boucles dont les itérations sont indépendantes.

3.2 Boucles parallélisées

Chaque case des tableaux est écrite une seule fois. Leur initialisation peut donc être répartie entre les threads :

```

1 #pragma omp parallel for
2 for (int i = 0; i < N; i++) {
3     for (int j = 0; j < N; j++) {
4         A[IDX(i, j)] = 0;
5         B[IDX(i, j)] = 0;
6     }
7 }

```

La boucle principale est parallélisée sur les lignes. À chaque itération, A reste en lecture seule pendant que les threads écrivent dans des lignes différentes de B.

```
1 for (int it = 0; it < ITER; it++) {
2     #pragma omp parallel for
3     for (int i = 1; i < N - 1; i++) {
4         for (int j = 1; j < N - 1; j++) {
5             /* Regle de propagation */
6         }
7     }
8
9     int *tmp = A;
10    A = B;
11    B = tmp;
12 }
```

Enfin, le comptage utilise une réduction. Chaque thread calcule des compteurs locaux, puis OpenMP les additionne dans c0, c1 et c2. Sans réduction, plusieurs threads pourraient modifier simultanément les mêmes variables.

```
1 #pragma omp parallel for reduction(+:c0,c1,c2)
2 for (int i = 0; i < N; i++) {
3     for (int j = 0; j < N; j++) {
4         if (A[IDX(i,j)] == 0) c0++;
5         else if (A[IDX(i,j)] == 1) c1++;
6         else c2++;
7     }
8 }
```

3.3 Compilation et exécution

```
gcc -fopenmp epidemie_omp.c -o epidemie_omp
./epidemie_omp
```

Le nombre de threads est fixé dans le programme :

```
omp_set_num_threads(8);
```

Le temps écoulé est mesuré avec `omp_get_wtime`.

3.4 Mesures obtenues

L'accélération, ou *speedup*, est calculée par $S(p) = T(1)/T(p)$, où p est le nombre de threads.

Le premier test utilise $N = 2000$, soit 4 000 000 de cellules, et ITER=500.

Threads	Temps	Accélération
1	9,333 s	1,000
2	4,998 s	1,867
4	6,076 s	1,536
8	3,003 s	3,108

Le second test utilise $N = 8312$, soit 69 089 344 cellules, et ITER=500. Cette taille est proche de la population française et sert ici uniquement d'ordre de grandeur.

Threads	Temps	Accélération
1	2 min 33 s	1,00
2	1 min 30 s	1,70
4	1 min 17 s	1,99
8	48 s	3,19

Dans les deux tests, huit threads donnent le meilleur temps. L'accélération n'est cependant pas proportionnelle au nombre de threads. Le résultat à quatre threads sur la petite grille est même moins bon qu'à deux threads. Cela peut s'expliquer par le coût de création et de synchronisation des threads, par le partage de la bande passante mémoire et par la variabilité des mesures.

4 Parallélisation avec Open MPI

4.1 Mémoire distribuée et initialisation

MPI (*Message Passing Interface*) permet de programmer plusieurs processus qui possèdent chacun leur propre mémoire. Open MPI est l'implémentation utilisée pour compiler et lancer le programme. Tous les processus exécutent `epidemie_mpi.c`, mais chacun travaille sur une partie différente de la grille.

```
1 MPI_Init(&argc, &argv);  
2 MPI_Comm_rank(MPI_COMM_WORLD, &rang);  
3 MPI_Comm_size(MPI_COMM_WORLD, &nbproc);
```

`MPI_Init` démarre l'environnement MPI. `MPI_Comm_rank` fournit le numéro du processus courant et `MPI_Comm_size` le nombre total de processus. Le communicateur `MPI_COMM_WORLD` contient tous les processus lancés. L'environnement est terminé par `MPI_Finalize`.

4.2 Découpage de la grille

La grille est découpée horizontalement en blocs de lignes :

```
1 int bloc = N / nbproc;  
2 int debut = rang * bloc;
```

Le processus de rang `rang` traite `bloc` lignes à partir de la ligne globale `debut`. Le programme impose que N soit divisible par le nombre de processus. Pour $N = 300$ et quatre processus, chaque processus traite 75 lignes.

Processus	Lignes globales
0	0 à 74
1	75 à 149
2	150 à 224
3	225 à 299

Les trois personnes initialement infectées sont placées uniquement par le processus qui possède leurs lignes globales.

4.3 Lignes fantômes

La première et la dernière ligne d'un bloc dépendent de cellules appartenant aux processus voisins. Chaque processus alloue donc deux lignes supplémentaires :

```
1 int *A = malloc((size_t)(bloc + 2) * N * sizeof(int));  
2 int *B = malloc((size_t)(bloc + 2) * N * sizeof(int));
```

- la ligne locale 0 reçoit la frontière du processus précédent ;
- les lignes 1 à `bloc` contiennent le bloc réellement calculé ;
- la ligne `bloc + 1` reçoit la frontière du processus suivant.

Ces lignes fantômes donnent accès aux voisins verticaux sans partager directement la mémoire des autres processus.

4.4 Échange des frontières

Avant chaque itération, les processus voisins échangent leurs lignes extrêmes avec `MPI_Send` et `MPI_Recv`.

```
1 MPI_Send(&A[IDX(bloc,0)], N, MPI_INT, rang + 1, 0,  
2         MPI_COMM_WORLD);  
3 MPI_Recv(&A[IDX(bloc + 1,0)], N, MPI_INT, rang + 1, 1,  
4         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Les communications sont organisées en deux phases : d'abord entre les paires 0–1, 2–3, etc., puis entre les paires 1–2, 3–4, etc. Dans chaque paire, l'ordre des envois et des réceptions dépend de la parité du rang. Cette organisation évite que deux processus exécutent en même temps un envoi bloquant en attendant l'autre.

Une fois les frontières reçues, chaque processus applique la règle de propagation à ses lignes locales. Les pointeurs A et B sont ensuite échangés localement, comme dans la version séquentielle.

4.5 Réduction des résultats

Chaque processus compte les états présents dans son bloc. Une réduction additionne ensuite les compteurs locaux sur le processus 0 :

```
1 MPI_Reduce(compteurs_locaux, compteurs_globaux, 3,
2           MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

Une seconde réduction est utilisée pour les temps locaux. Seul le processus 0 affiche les résultats, ce qui évite plusieurs affichages identiques.

4.6 Compilation et exécution

```
mpicc epidemie_mpi.c -o epidemie_mpi
mpirun -np 4 ./epidemie_mpi
```

L'option `-np 4` lance quatre processus. Le découpage actuel impose que le nombre de processus divise N .

5 Analyse comparative

5.1 Différences entre OpenMP et MPI

Caractéristique	OpenMP	MPI
Unité parallèle	Thread	Processus
Mémoire	Partagée	Distribuée
Répartition	Boucles	Blocs de lignes
Communication	Variables communes	Messages
Résultat global	Réduction OpenMP	MPI_Reduce

OpenMP demande peu de modifications, car les threads accèdent directement à la grille complète. MPI nécessite un découpage explicite des données, des lignes fantômes et des communications entre processus. En contrepartie,

MPI peut fonctionner sur plusieurs machines, tandis qu'OpenMP est limité à la mémoire partagée d'une machine.

5.2 Validation et limites des mesures

La première vérification porte sur les compteurs finaux. À paramètres identiques, les trois versions doivent produire le même nombre de personnes saines, infectées et guéries. Cette comparaison permet de détecter une erreur de synchronisation, de réduction ou d'échange de frontière.

Les tableaux précédents permettent d'analyser OpenMP, mais ils ne permettent pas encore de comparer directement OpenMP et MPI. Les mesures OpenMP ont été réalisées avec 500 itérations et des grilles de taille 2000 ou 8312, tandis que la version MPI utilise par défaut $N = 300$ et 200 itérations. Une comparaison équitable exige la même grille, le même nombre d'itérations, la même machine et plusieurs répétitions pour chaque nombre de threads ou de processus.

La mesure du temps MPI pourrait également être améliorée. Le programme utilise `clock()` et affiche la moyenne des temps locaux. Pour représenter la durée réellement perçue, il serait préférable de mesurer le temps écoulé et de retenir le temps du processus le plus lent, puisque l'exécution globale dépend de sa terminaison.

6 Conclusion

Le programme séquentiel se parallélise naturellement parce que toutes les cellules d'une itération lisent l'ancien tableau **A** et écrivent dans le nouveau tableau **B**. OpenMP exploite cette indépendance en répartissant les lignes entre plusieurs threads et en utilisant une réduction pour les compteurs. Les mesures montrent un gain de temps, avec une meilleure performance observée à huit threads, mais une accélération non linéaire.

La version MPI applique la même règle de calcul dans un modèle à mémoire distribuée. La grille est découpée en blocs horizontaux et les processus échangent leurs frontières avant chaque itération. Les résultats locaux sont ensuite réunis avec `MPI_Reduce`.

Les deux approches répondent donc au même objectif avec des modèles différents. La suite logique du travail consiste à appliquer un protocole de mesure commun aux trois versions afin de comparer leurs performances sans changer la taille du problème.

7 Références

- Thierry Garcia, *Initiation au calcul parallèle intensif*, supports de cours CM4 (OpenMP) et CM5 (MPI), CY Tech, 2026.
- INSEE, données démographiques utilisées comme ordre de grandeur pour la population française : [insee.fr](https://www.insee.fr).